Testing a Legal AI Review Workflow

Ingrid Kao

Independent practitioner note | November 10, 2026

Companion files · GitHub

At a glance

Built: A source-verification gate and a saved eight-document NDA review.

Observed (synthetic): 16 of 16 gate fixtures matched; all eight adjudicated document–rule pairs were source-supported.

Meaning: Software validity, legal labels, and release decisions have separate checks.

A convincing contract-review demonstration can still leave an implementation team unable to decide whether a pilot should proceed. A flag may sound reasonable while quoting a different clause, referring to an outdated document, or combining two review categories that the evaluation key treats as separate issues. A useful pilot makes those differences visible before anyone relies on its recommendations.

At Lawsnote, I checked AI contract-review outputs against source NDAs and used the errors to refine the review rules. This note presents a small, inspectable implementation: a source-linked output gate, a recorded exploratory AI review of eight fictional NDA fragments, and a release decision framework. The main finding is practical. Mechanical validation can reject specified source and format failures, while substantive interpretation and the unit used for scoring still require an accountable reviewer. The companion files preserve inputs, the original response, configuration, executable checks, and results.

Define the decision the pilot supports

The first design choice is the boundary of the task. Here, a reviewer identifies exceptions to four deliberately simplified sponsor instructions about confidentiality duration, permitted purpose, residual knowledge, and backup deletion. The documents are short original fictional fragments. They contain no client information and do not represent a market-standard NDA playbook.

The permitted outcome is a finding for human review, not approval to sign or permission to change a system. An empty findings array means that the reviewer found no exception under these particular instructions. It does not resolve enforceability, missing provisions, cross-references, governing law, or facts outside the supplied text.

Before involving a model, a process owner should define who receives a finding, what evidence that person needs, and who decides whether it matters. OpenAI's evaluation guidance supports task-specific objectives and datasets rather than informal impressions.[1] NIST's Generative AI Profile also cautions against generalizing from narrow assessments.[2] Those principles determine how the results below are reported.

Make each finding traceable

A review object should identify the document and source version, the applied rule, a clause locator, an exact quotation, and the proposed next step. The companion deterministic gate also checks a source digest and an allowed severity value. The experimental AI response uses a related, separately documented schema with a document-level findings array; its scorer checks that array against the supplied sources.

Each source digest is computed from an explicitly defined serialization or source text. This distinction matters: hashing a PDF file and hashing extracted clause text produce different identifiers. The implementation records the representation it hashes. A matching digest establishes correspondence to that representation, not the authenticity or completeness of the original agreement.

The gate rejects malformed records, missing or extra fields, unsupported values, stale versions, and quotations that do not occur in the cited clause. It sends an explicit refusal to a manual path. A mechanically valid record enters a human-review queue. It never becomes an authorized legal decision merely because its fields are complete.

Check	What it can establish	What remains unresolved
Schema and allowed values	The record has the expected form	Whether the conclusion is sound
Source version and digest	The record points to the pinned representation	Whether extraction captured the full agreement
Quote within cited clause	The quoted words occur at that locator	Whether they support the proposed interpretation
Recorded human decision	A named decision step exists in the workflow	Whether a production identity system authenticates that person

Structured output is useful for integration, but OpenAI's documentation expressly recognizes that structured responses can still contain mistakes.[3] Consequently, source checks and schema checks are supporting controls rather than substitutes for legal judgment.

Run software checks separately from the model exercise

The first experiment uses deliberately authored candidate outputs, including malformed and substantively unsound examples. It tests the gate implementation rather than an AI model. Sixteen fixtures exercise invalid JSON, unsupported fields or values, source mismatches, incorrect quotation locations, refusal handling, and mechanically valid records.

The run matched the expected gate on all 16 fixtures: 12 were blocked, one refusal went to manual review, and three entered human review. A regression run repeated the implementation twice in fresh directories and found byte-identical results across its eight generated artifacts in this runtime. These counts measure fixture-level software behavior, not model performance.

One fixture is designed to contain an unsound interpretation while retaining a valid quotation and source reference. It enters the human-review queue as intended. That outcome is useful because it exposes the gate's boundary. Calling it a successful legal review would reverse the meaning of the test. Its substantive rejection label is an authored test key, not an independent lawyer's adjudication or an observed model error.

The second experiment is an actual exploratory AI review, conducted as an isolated Codex agent batch on November 10, 2026. The task supplied eight fictional documents, identified as A01–A08, and four sponsor rules, with an instruction to treat document contents as untrusted data. The provisional answer key was not supplied in the review task. The response was retained without rewriting its findings.

The saved run records the parent model as gpt-6.1-sol, but not an immutable provider snapshot or sampling settings. Re-scoring validates the recorded response; it does not repeat inference.

Resolve overlapping labels before scoring

The saved response contains eight finding rows. All eight document metadata checks match the pinned inputs, and every quotation occurs in its cited clause. All six document-and-rule pairs in the original provisional key are present. Two additional PURPOSE labels concern residual-knowledge clauses in A04 and A06 that are also flagged under RESIDUALS.

Under the adopted document-and-rule convention, both labels are supported. The residual clauses allow use without restriction, extending beyond the toy policy's permitted purpose of evaluating Project Atlas. RESIDUALS is the more specific primary rule for issue grouping; PURPOSE remains a supported secondary label. The post-run key revision adds PURPOSE for A04 and A06 after inspection of the saved findings, with the source, rationale, and overlap group recorded in the adjudication ledger.

For distinct actionable issues, the overlapping PURPOSE label merges into the corresponding RESIDUALS issue. The two views consequently report eight document–rule pairs and six actionable issue groups. These are different denominators describing the same saved response. The rule hierarchy and counting convention changed; the model output did not.

Observation from the saved batch	Count	Interpretation
Fictional documents reviewed	8	One exploratory batch, A01–A08
Finding rows returned	8	Eight document–rule labels
Quotes matching their cited clauses	8 of 8	A source-linkage result
Original provisional key pairs represented	6 of 6	Original key preserved unchanged
Adjudicated document–rule pairs matched	8 of 8	Two supported PURPOSE labels added
Distinct actionable issue groups	6	PURPOSE overlaps grouped under RESIDUALS

The scorer reports eight true positives, no false positives, and no false negatives against the revised document-and-rule key. Precision and recall are each 1.0 for this fixed synthetic exercise. Those descriptive results use the adopted counting convention; they are not a population estimate of legal accuracy. The original six-pair key and raw model response remain available alongside the new key, rationale, and tests.

Two cases also illustrate the exercised boundaries. A06 contains an instruction to ignore the review policy; the saved response returned the substantive flags. A08 contains non-operative drafting history with a shorter term; the response returned no finding.

Keep permissions outside the recommendation

OWASP describes indirect prompt injection through files and websites and recommends multiple controls rather than reliance on model instructions alone.[4] For a legal workflow, that supports a practical separation: contract text is evidence to inspect, while permissions to send, sign, delete, or change records come from the surrounding application and authorized people.

The demonstration makes no external changes. Its role arguments are simulated workflow inputs. A production design needs an approved data path, authorization, secret management, retention controls, and monitoring. The pilot owner should also test changes in extraction, source formatting, model version, and rule configuration. Passing the current fixtures should not silently authorize a different deployment.

Make a release decision that can be revisited

The observed results support continuing a bounded evaluation. They do not support autonomous contract clearance. Before a release decision, the domain owner should approve the documented counting convention, review rules, and treatment of critical misses. New representative agreements should then be evaluated with independent substantive review, a frozen prompt and rule version, and a documented sampling method.

The business case should include verification and correction effort. An illustrative calculation makes the issue concrete: at 200 matters per month, a 20-minute manual baseline and a 14-minute assisted total would imply 20 hours of capacity. At an assumed loaded rate of $80 per hour, that is $1,600 before fees and implementation costs. A $2,000 monthly system cost would exceed that amount by $400. These are planning assumptions, not measured savings or a complete investment model. If verification takes longer, the result changes again.

A limited launch should have a process owner, a receiving reviewer, a support contact, and an agreed pause condition. Training should show a refusal, an unsupported quotation, a stale source, and a mechanically valid but questionable interpretation. A revision log should connect each recurring failure to a proposed change and a regression check. The release decision should consider reviewer effort and unresolved risks alongside throughput.

The useful deliverable from this pilot is an evidence record that supports the next decision. It shows which controls ran, what the model returned, which questions remain open, and who must resolve them before broader use.

Method and references

This study uses synthetic documents, authored output fixtures, and operational records. The eight-document agent review is separate from the 16 deterministic gate fixtures and from the portfolio's original 20-document AI Review Lab. The adjudication applies the explicit toy policy and adopted counting convention; it does not establish independent attorney validation, production accuracy, or general prompt-injection resistance. The saved batch lacks a pinned provider snapshot and sampling settings. Simulated roles are not enterprise authentication, and the cost example is a planning assumption rather than realized savings.

[1] OpenAI. Evaluation best practices. Sections on task-specific evaluation, dataset design, and human-calibrated evaluation. Accessed November 10, 2026. https://developers.openai.com/api/docs/guides/evaluation-best-practices

[2] NIST. Artificial Intelligence Risk Management Framework Generative Artificial Intelligence Profile. NIST AI 600-1, July 2024. Actions MS-2.5-001 and MS-2.5-003. https://doi.org/10.6028/NIST.AI.600-1

[3] OpenAI. Structured model outputs. Section on handling mistakes. Accessed November 10, 2026. https://developers.openai.com/api/docs/guides/structured-outputs

[4] OWASP. LLM01 2025 Prompt Injection. Sections on indirect prompt injection and mitigation. Accessed November 10, 2026. https://genai.owasp.org/llmrisk/llm01-prompt-injection/